Lifelong Mapping

1. Course Content

- Run the sample program to build a grid map and a pose graph format map, and then restore the previous mapping state using the pose graph to continue mapping.

2. Function Introduction

- The Lifelong Mapping feature is accomplished using the slam_toolbox algorithm to build a pose graph. It is designed to support robots in continuously building, updating, and maintaining maps in long-term, dynamic environments. Its features include:
    - **Map Persistence and Reuse**: Supports resuming mapping after an interruption by storing and reloading pose-graph data through serialization and deserialization.
    - **Incremental Map Updates**: Continues to expand on an existing map, updating for environmental changes (such as new objects or temporary obstacles). This is very important for mapping large scenes like warehouses, as it avoids having to start from scratch every time the environment changes.
- Application Scenarios
    - In scenarios like warehouses and shopping malls, robots need to perform long-term patrol and transport tasks. Lifelong mapping can dynamically update the map to cope with changes like moved shelves and temporary obstacles, while retaining valid historical data to avoid re-mapping.
    - When a robot needs to pause due to low battery or task interruption, it can resume mapping by loading the existing pose graph upon restart, without starting from scratch. For example, a campus inspection robot can complete a large-area map in different time segments.

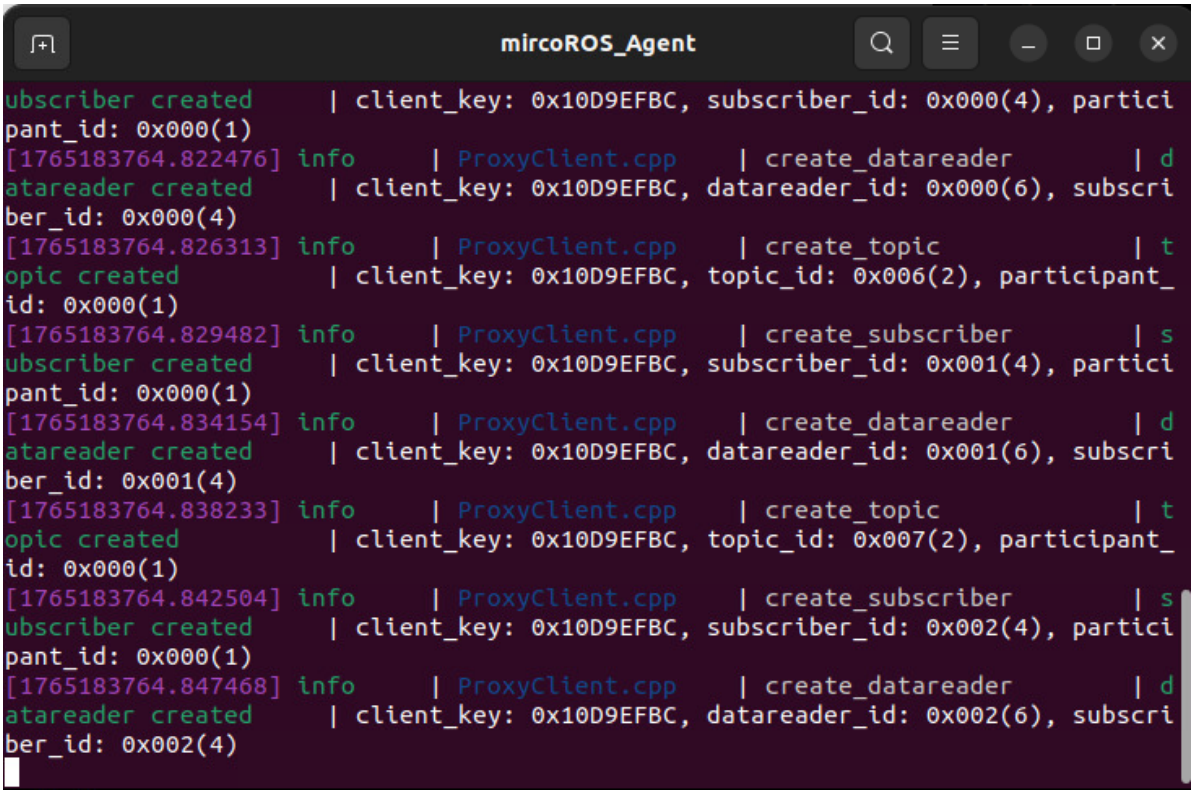
3. Start the Agent

Note: If it is already running, you do not need to start it again.

Enter the command in the vehicle's terminal:

```
start_agent
```

The terminal will print the following information, indicating a successful connection:



4. Build the Map

4.1 Run the Mapping Node

- Launch the mapping command in the vehicle's terminal:

For Raspberry Pi 5 users, if the ROS container is not started, you need to start it first,

```
bringup_m3
```

Open a terminal inside the container,

```
exec_m3
```

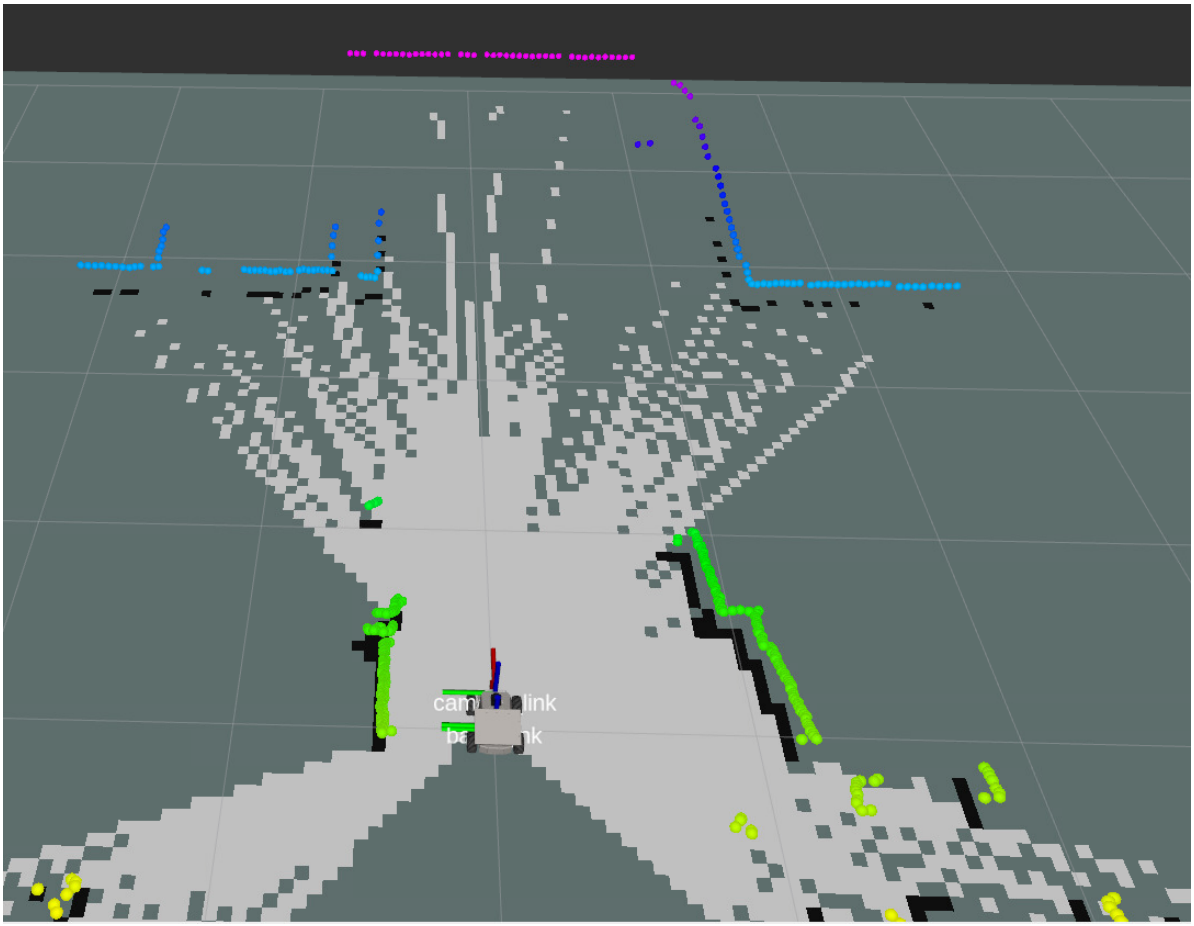
(ORIN)

```
ros2 launch m3_bringup slam_toolbox.launch.py
```

(RDK & PI5 boards) Run in separate steps, with visualization launched on the accompanying virtual machine `Rosmaster-M3-Ubuntu22.04`,

```
# On the virtual machine
rviz2 -d
/home/yahboom/yahboomM3_ws/user_ws/src/m3_bringup/rviz/slam_toolbox_rviz.rviz
# On the host machine
ros2 launch m3_bringup slam_toolbox.launch.py gui:=False
```

> [!NOTE]
>
> - You can use the shortcut command `slam` to start mapping.

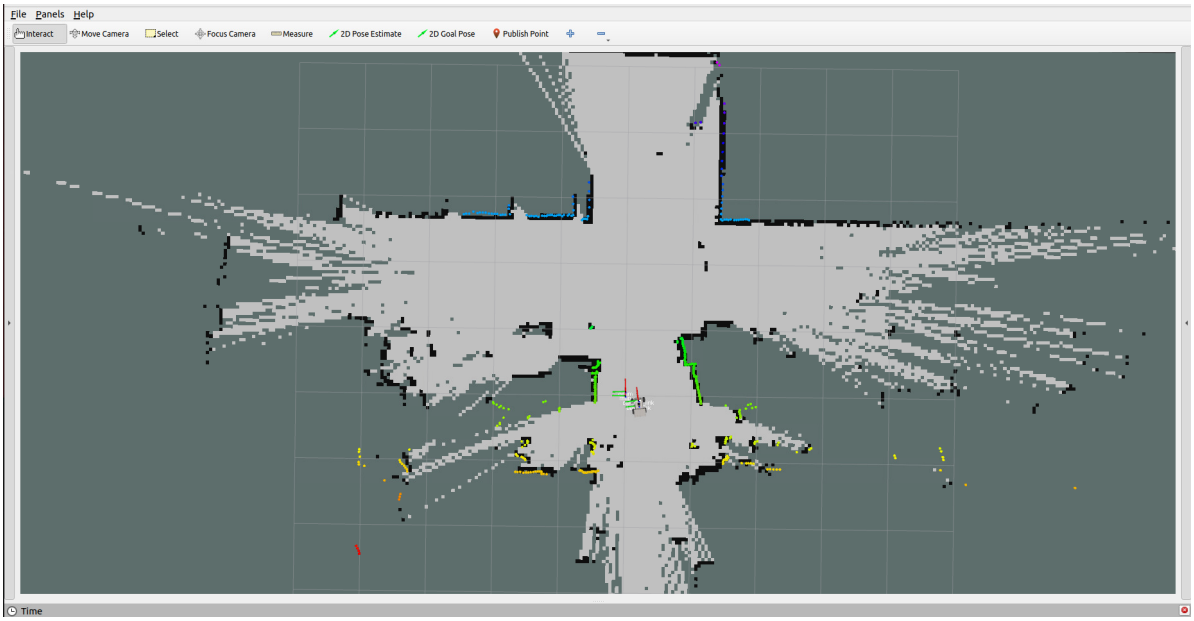


- To control the vehicle's movement, you can use either keyboard or joystick control. Here, we use keyboard control as an example:

```
keyboard_control
```

Click on the terminal window with the mouse and press `z` to reduce the speed appropriately.

- Press `I`, `<`, `J`, `L` to control the car to move forward, backward, turn left, and turn right, respectively, and press `k` to stop. Control the car to move slowly to complete the mapping.



4.2 Save the Pose Map

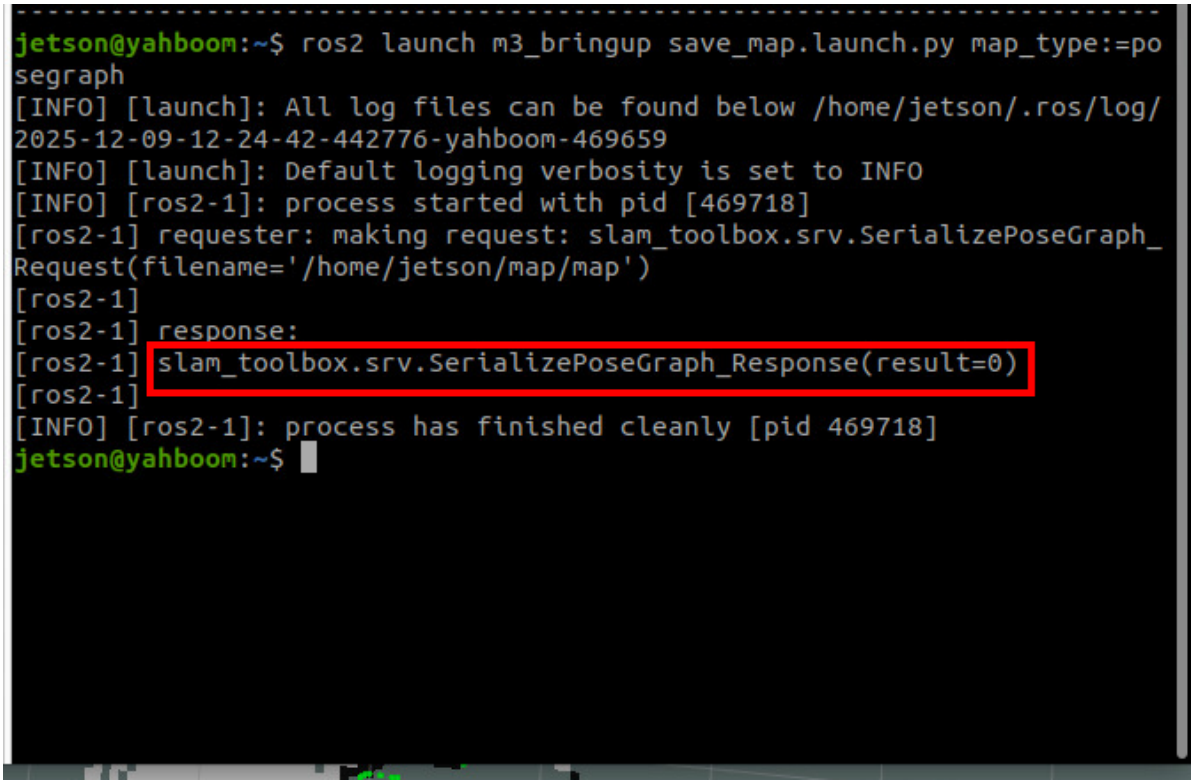
4.2.1 Method 1: Save via Command Line

- Run the command in the terminal:

```
ros2 launch m3_bringup save_map.launch.py map_type:=posegraph
```

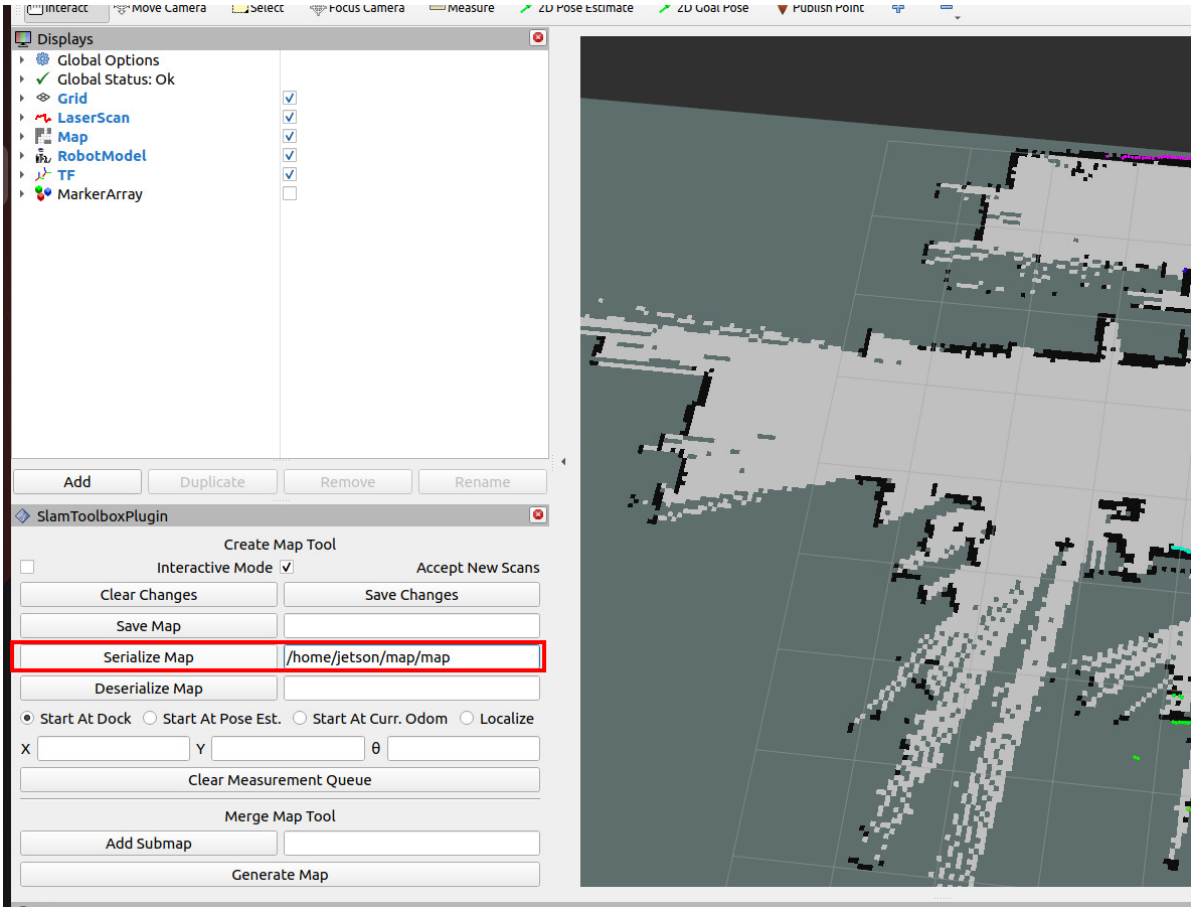
> [!NOTE]
>
> Optional launch parameters:
>
> - `map_name`: The name of the pose map to be saved, default is `map`.

- The terminal prompt `result=0` indicates that the pose map was saved successfully.

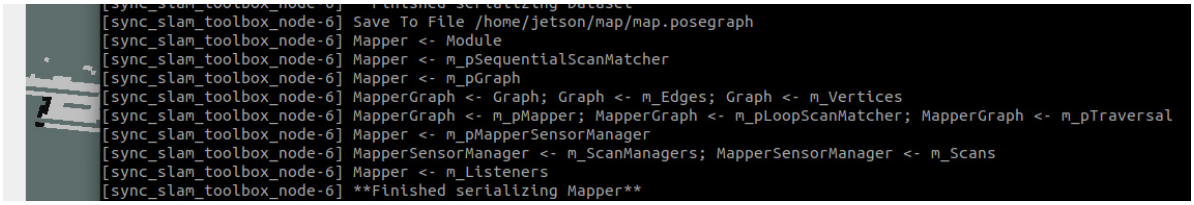


4.2.2 Method 2: Save via Panel

- In the blank space after `Serialize Map`, enter the path and name for the pose map to be saved, then click `Serialize Map` to serialize the map.



- The terminal will display the following message to indicate that the pose map has been saved.



4.3 Save the Grid Map

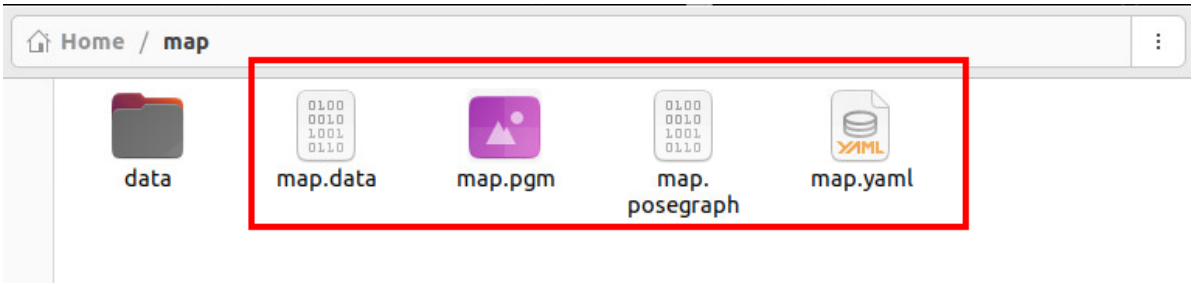
- Run the following command in the terminal to save the map:

```
ros2 launch m3_bringup save_map.launch.py
```

The terminal prompt **Map saved successfully** indicates that the map was saved successfully.

```
jetson@yahboom:~$ ros2 launch m3_bringup save_map.launch.py
[INFO] [launch]: All log files can be found below /home/jetson/.ros/log/2025-12-09-09
-38-25-740040-yahboom-42028
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [map_saver_cli-1]: process started with pid [42079]
[map_saver_cli-1] [INFO] [1765244306.041476476] [map_saver]:
[map_saver_cli-1]       map_saver lifecycle node launched.
[map_saver_cli-1]       Waiting on external lifecycle transitions to activate
[map_saver_cli-1]       See https://design.ros2.org/articles/node_lifecycle.html for
more information.
[map_saver_cli-1] [INFO] [1765244306.041784604] [map_saver]: Creating
[map_saver_cli-1] [INFO] [1765244306.042322941] [map_saver]: Configuring
[map_saver_cli-1] [INFO] [1765244306.050261831] [map_saver]: Saving map from 'map' to
pic to '/home/jetson/map/map' file
[map_saver_cli-1] [WARN] [1765244306.069662081] [map_io]: Image format unspecified. S
etting it to: pgm
[map_saver_cli-1] [INFO] [1765244306.071145443] [map_io]: Received a 225 X 453 map @
0.05 m/pix
[map_saver_cli-1] [INFO] [1765244306.129794768] [map_io]: Writing map occupancy data
to /home/jetson/map/map.pgm
[map_saver_cli-1] [INFO] [1765244306.131996019] [map_io]: Writing map metadata to /ho
me/jetson/map/map.yaml
[map_saver_cli-1] [INFO] [1765244306.132354356] [map_io]: Map saved
[map_saver_cli-1] [INFO] [1765244306.132412180] [map_saver]: Map saved successfully
[map_saver_cli-1] [INFO] [1765244306.135179544] [map_saver]: Destroying
[INFO] [map_saver_cli-1]: process has finished cleanly [pid 42079]
jetson@yahboom:~$
```

- The saved grid map and pose map are in the `~/map` directory on the system, where:
  - `map.data` and `map.posegraph` are the pose map files.
  - `map.pgm` and `map.yaml` are the grid map files.



- Then press `Ctrl+C` to end the mapping.

5. Restore Mapping State

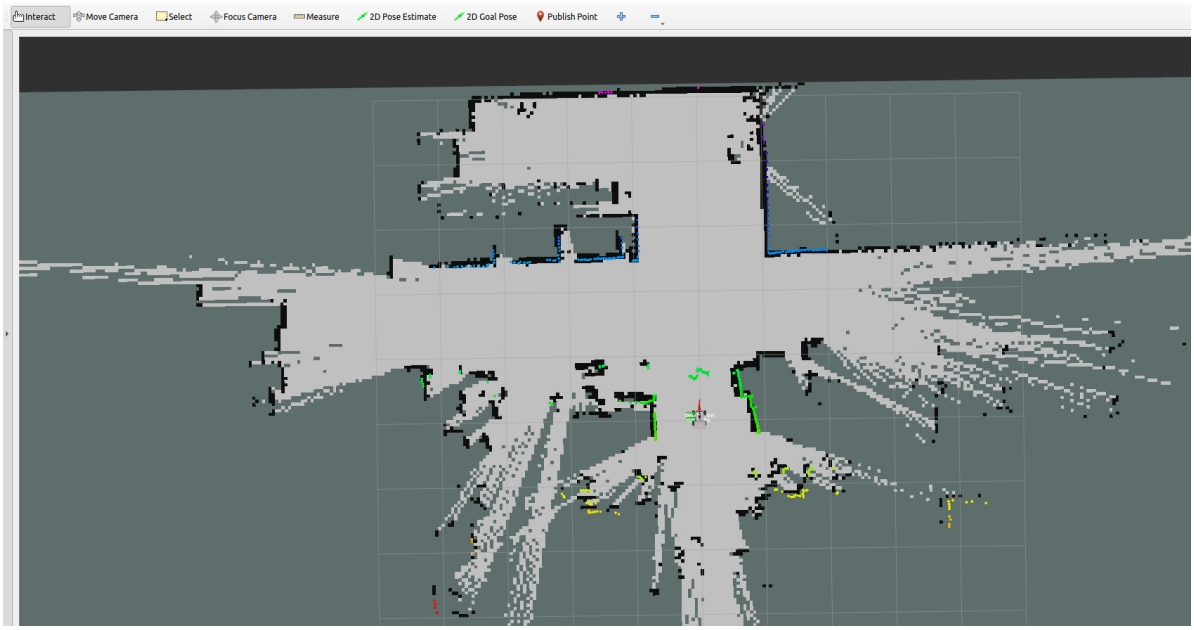
5.1 Restore from the Mapping Origin

- If the vehicle is at the original starting point of the mapping, restore the mapping state with the following command:

```
ros2 launch m3_bringup slam_toolbox.launch.py pose_graph:=map
```

> `pose_graph:=map` is the name of the pose map being passed in.

- You can then continue building or updating the map from the previously built map state.

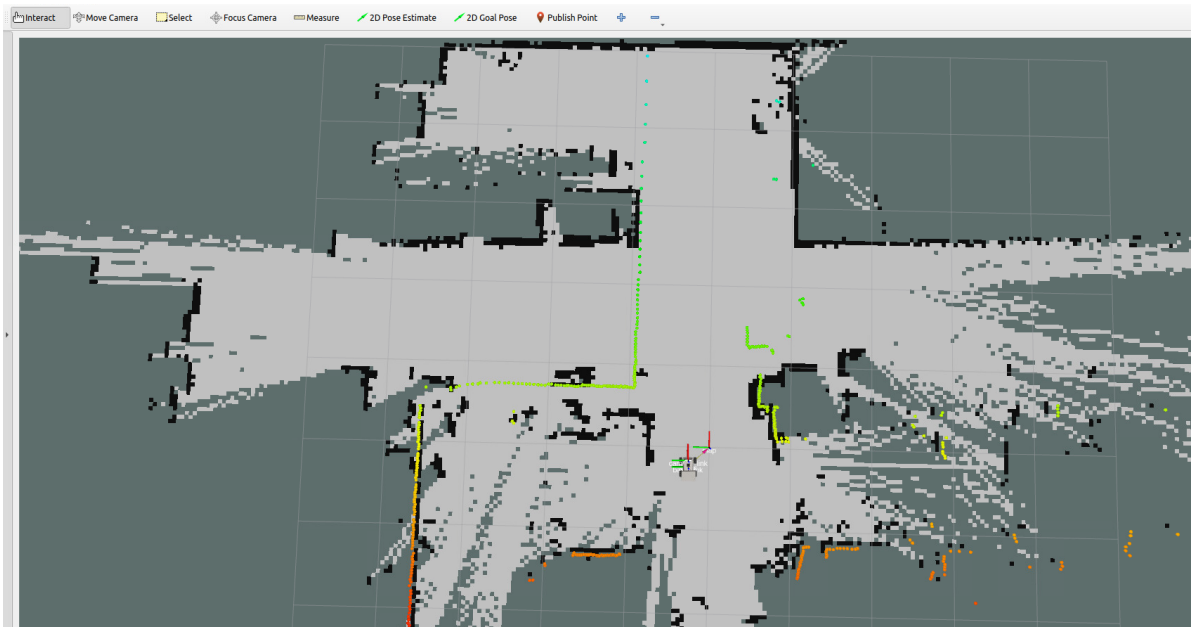


5.2 Restore from Any Point on the Map

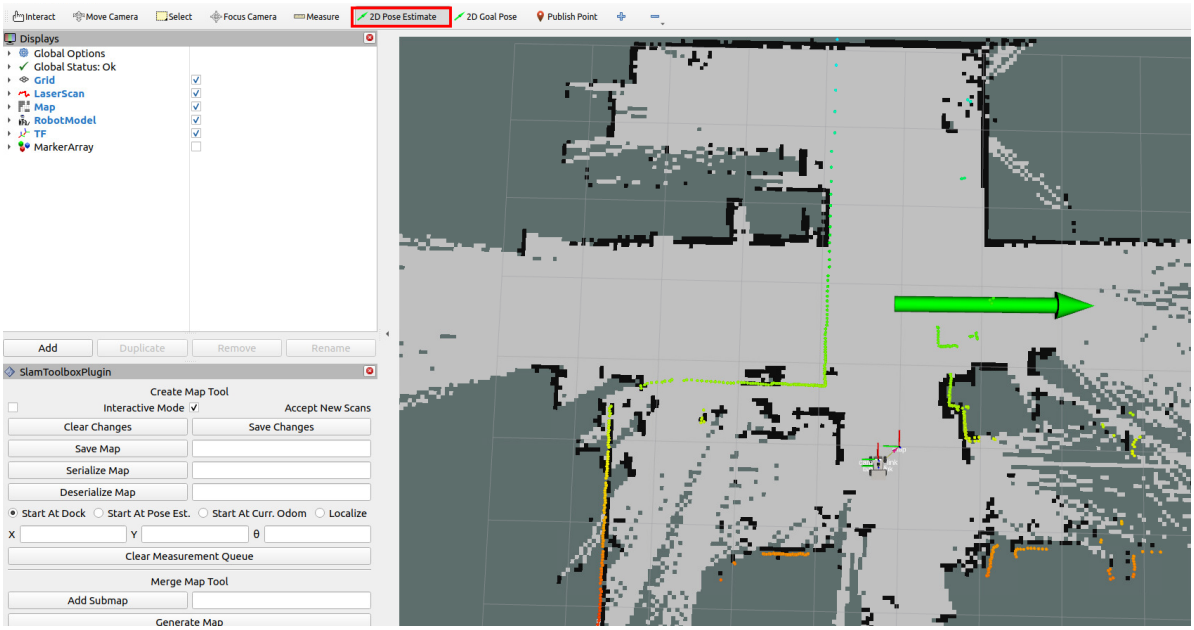
- If the robot is at any position on the map, first start the mapping node:

```
ros2 launch m3_bringup slam_toolbox.launch.py pose_graph:=map
```

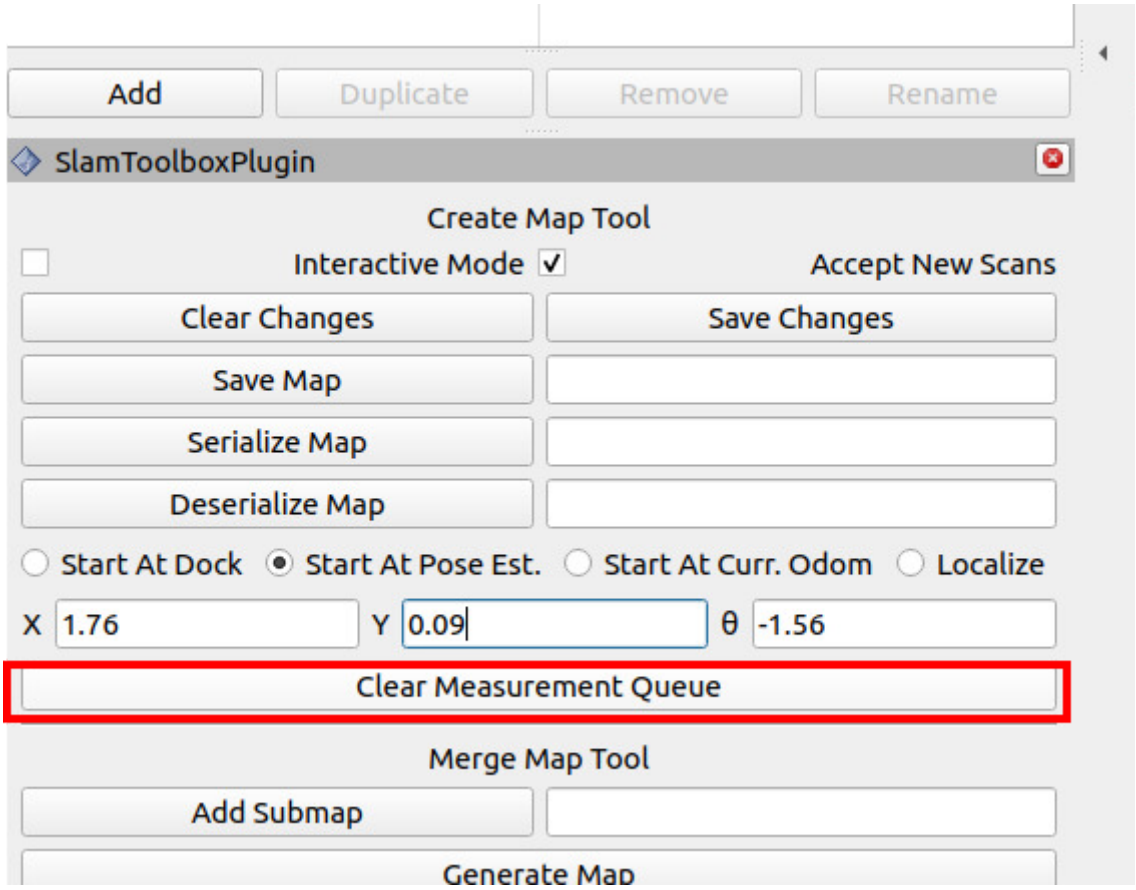
- As you can see, if the robot is not at the map's origin, the laser point cloud and the map environment features will not match.



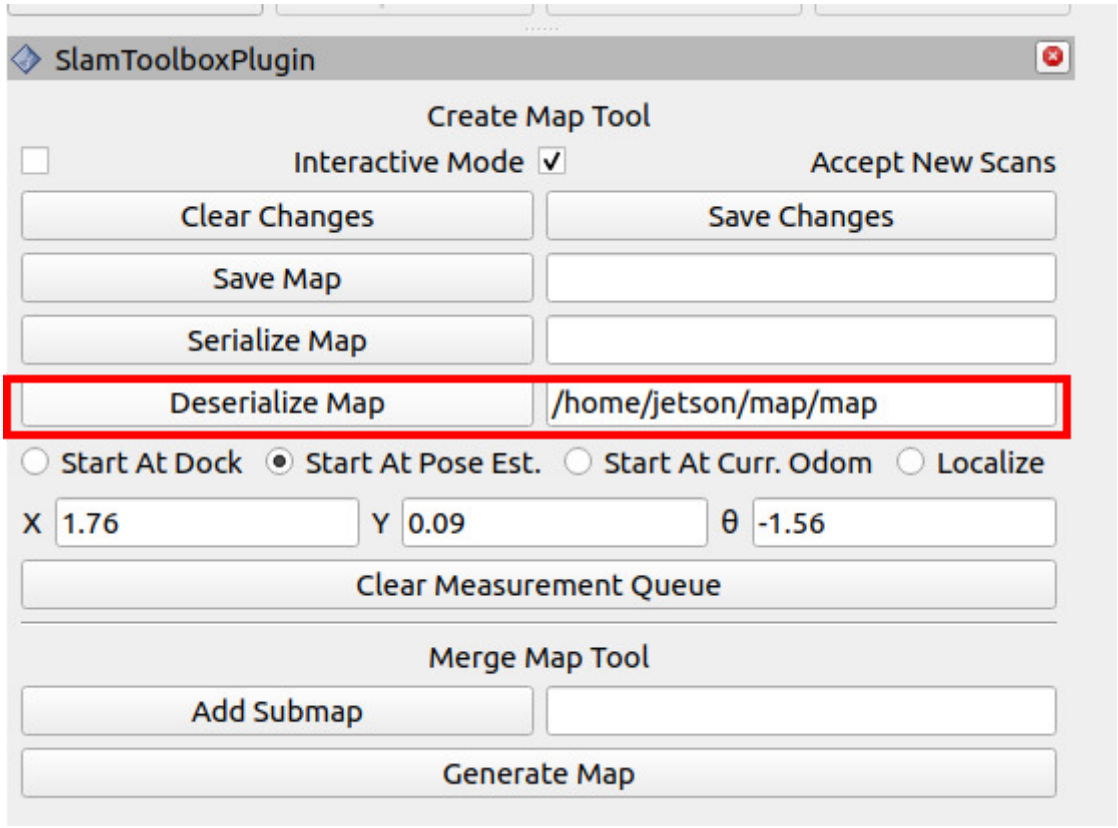
- First, use the `2D Pose Estimate` tool to give an approximate pose on the map.



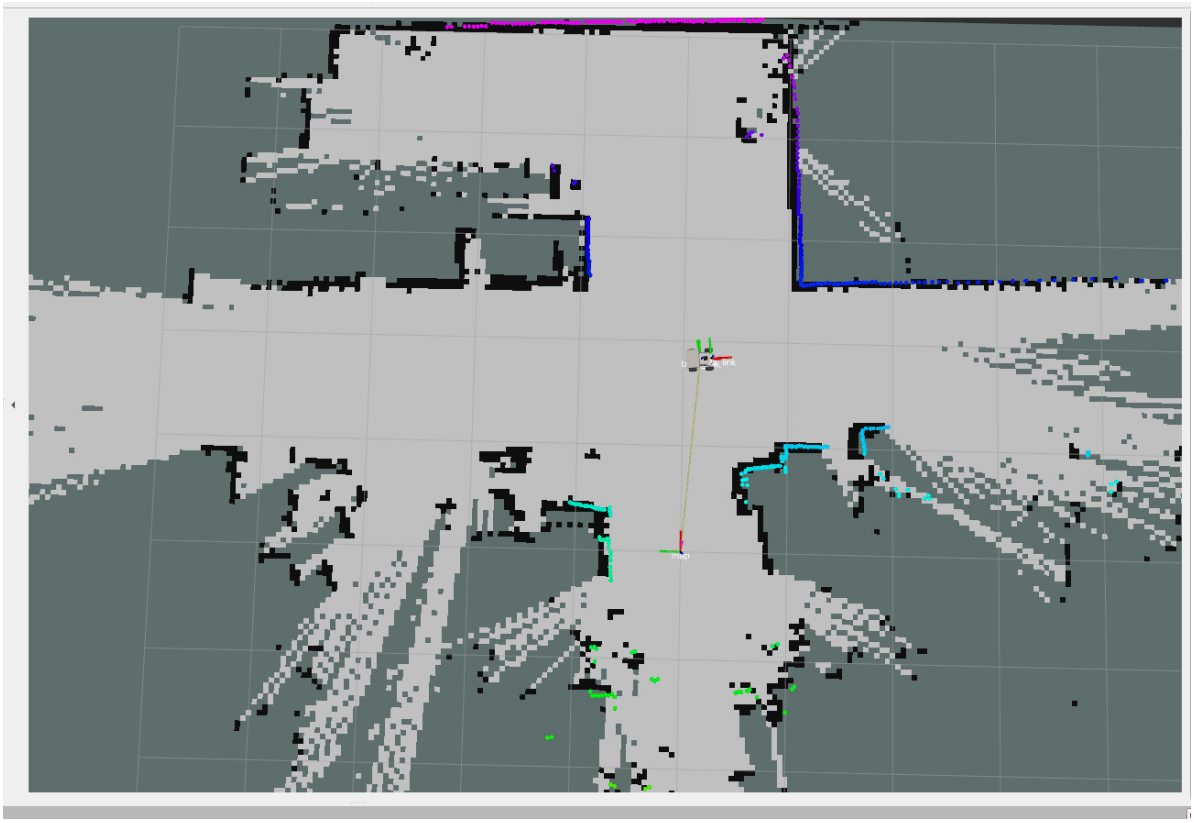
- In the left toolbar, click `Clear Measurement Queue`, and the current pose data will be automatically filled in.



- Then, in the options bar next to `Deserialize Map`, enter the path of the previously saved pose map (note: do not include the file extension).
- Then click `Deserialize Map` to deserialize the map.



- The robot has restored its correct pose in the global map, and you can now continue building or updating the map from the previously built state.



5.3 Save Mapping Progress

- You can also save pose maps with different names at various stages of the mapping process, similar to game saves, to restore the mapping state from different points of progress.